

# Unwoven Harmony

---

## Unwoven Harmony (*student proposal*)

---

**Assessment:** ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

**Proposal (as submitted):** An action game where **gesture-based combat** (tap, press-and-hold, swipe) meets **dynamic music** inspired by Chinese mythology. Players cast elemental magic to defeat enemies and bosses across short, music-driven stages. Unlike traditional rhythm games, hitting the beat is **optional but rewarding** — you can clear a stage by mashing, but hitting on beat grants special finishing moves and extra visual/audio flair. The soundtrack fuses traditional Chinese instruments with modern electronic beats; before a stage, players customise a moveset by equipping up to three of the **Five Elements**, and can **import their own music**. Named technical features: JSON data serialization for easy reading/manual editing; **in-built BPM recognition** for custom tracks; **enemy AI that conforms to the track's BPM**; and **server-side data handling to prevent cheating**. Loop: a C++ game loop handling a large queue of gestures for player + enemy attacks/movesets. Tool: a **music importer** that lets users add their own music, automatically matching attacks to the beat. Design features: fullscreen gesture combat (tap/hold/swipe anywhere, mapped to the Five Elements — Wood, Fire, Earth, Metal, Water, each with unique properties); a **combo system** where on-beat inputs chain moves and build Qi Energy to trigger finishing moves (missing beats has no penalty); **5 linear PVE stages** (~2 min each, 1–2 enemies at a time, some/all with a boss, clear before the timer, enemies spawn on fixed timers independent of the beat); and an **ability-unlock system** (progression + a post-stage shop where stage-earned currency buys moves, cosmetics and music tracks).

### *References:*

- [Hi-Fi RUSH on Steam](#) (the defining precedent: an action-combat game where everything syncs to the music and hitting the beat **enhances** your combos but is never required — exactly the "optional but rewarding" hook)

- Metal: Hellsinger on Steam (on-beat attacks deal more damage and fill a Fury meter — the direct precedent for the on-beat-builds-Qi / finishing-move reward)
- AudioSurf on Steam (a game built around **importing your own music** and auto-generating beat-timed gameplay from any track — the custom-music-import + BPM-analysis precedent)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

| Core Tech Requirement                                       | Unwoven Harmony |
|-------------------------------------------------------------|-----------------|
| External Database                                           | ✓               |
| Authentication »                                            | ✓               |
| Analytics »                                                 | ●               |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓               |
| Application Communication                                   | ✓               |
| Continuous Integration                                      | ✓               |
| Data-Driven Architecture                                    | ✓               |
| Front-end + Back-end                                        | ✓               |
| Custom Tools Development                                    | ✓               |
| 2D Graphical Output                                         | ✓               |
| 3D (strong students only)                                   | ✕               |
| Networking System                                           | ●               |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile (fullscreen touch gesture combat + on-device music import suit a phone; device: Pixel 9a) — the Native PC build is the development/tools vehicle (the music-importer tool). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for the **\*\*server-side anti-cheat\*\*** data handling, accounts and the currency/unlock/leaderboard store.

**Front end:** A 2D renderer of the elemental combat (player, 1–2 enemies, bosses, per-element spell VFX), the fullscreen gesture-input layer, the Qi/

combo/timer HUD and the beat/on-beat flair, plus the shop and music-import screens. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES drawn in a GLSurfaceView for the realtime 2D; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

**Back end:** The C++ engine core (fixed-timestep loop over a large gesture queue for player + enemy movesets, the Five-Elements combat, the audio/BPM analysis feeding the combo/Qi and the BPM-conforming enemy AI) + a server-side data layer that validates progression/currency to prevent cheating.

**Languages:** C++ (engine); Firebase-backed services on the Extended tier.

**Backend scope:** Local now, server-ready (T2) — the anti-cheat / accounts / store seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — a fixed-timestep loop resolves a **large queue of gestures** into player and enemy moves, runs the Five-Elements combat, and drives the audio/BPM analysis that feeds the on-beat combo/Qi rewards and the BPM-conforming enemy behaviour.
- *Data-driven:* moves, the Five Elements and their properties, enemy types/patterns, stage layouts and shop/unlock data are all JSON (easy to read and hand-edit), so a new move or enemy is a data change.
- *2D:* sprite/particle rendering of the elemental combat, spell VFX and the on-beat flair + HUD (custom-rendered). *3D:* not needed. *Networking:* no real-time multiplayer; the anti-cheat / accounts / store calls are async client↔server (Application Communication ✓), so a real-time Networking System is optional (●).
- *Custom tool:* the **music importer** — import a track, run **BPM recognition**, and auto-generate the beat map so the player's attacks and the enemy AI align to the imported song.
- *External DB:* accounts, currency/unlocks/cosmetics and (server-validated) progression. *Auth »:* accounts for the anti-cheat store. *Analytics »:* optional — on-beat engagement and stage drop-off for tuning.

*Audio note:* audio here is **load-bearing** (the combat, combos and enemy AI all key off BPM/beat), so the BPM-recognition + adaptive-music tech is real CS work (M1G6 / M2G9), and the TEAM/Audio goals carry a sync clause (below).

*Monetization note:* the shop uses **stage-earned currency** for moves, cosmetics and tracks — progression, not pay-to-win. If real-money purchases are ever

added, keep them to the module's clean-monetization rule (cosmetics/tracks only, no pay-to-win).

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep loop (C++17/20) reading fullscreen tap/hold/swipe gestures into a large gesture queue and resolving player + enemy moves; CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G6 audio/BPM and M1G8 enemy AI run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component model in C++ with moves, the Five Elements and their properties, enemy types/patterns, stage layouts and shop/unlock data in hand-editable JSON, hot-reloadable — a new move or enemy is a data change.
- **Prerequisites:** G1(loop & entity update hooks).
- **Feeds into:** M2G2 (the importer tool + config), M1G8 (enemy data), M2G13 (unlock economy).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (gesture queue, beat/BPM state, Qi), Tracy profiling to hold the frame budget with a large gesture queue + combat VFX, Catch2 tests for gesture parsing, BPM detection and data loading, gated by a GitHub Actions Windows+Android matrix.
- **Prerequisites:** G1 (loop to instrument).

- **Feeds into:** M2G3 (CI hardening); the on-device budget M2G1 validates.
- **G4 2D Graphics** — the elemental-combat renderer (player, enemies, bosses, per-element spell VFX, HUD) is the whole read of the game.  
Tech & dependencies
  - **Tech: Web build:** Sprite/particle renderer with a camera, batching, per-element spell VFX and the Qi/combo/timer HUD + on-beat flair overlay; SDL2/3/OpenGL ES, GLSL, stb\_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** G1 (loop & window).
  - **Feeds into:** M2G4 (advanced spell/on-beat VFX); every visible frame.
- **G6 Audio System** — playback, mixing and event triggers **plus the in-built BPM recognition / beat analysis** that the whole game keys off.  
Tech & dependencies
  - **Tech:** FMOD or OpenAL playback + mixing, beat/BPM detection (onset/tempo analysis) exposing a beat clock to the gameplay, and event-triggered SFX for casts, hits and on-beat flair.
  - **Prerequisites:** G1 (loop & audio callback), G2 (move/audio data).
  - **Feeds into:** The combo/Qi on-beat reward, M1G8 (BPM-conforming enemy AI), M2G2 (the music importer), M2G9 (adaptive music).
- **G8 Enemy AI & Pathfinding** — the enemy roster whose behaviour **conforms to the track's BPM**, with fixed-timer spawns for fair non-rhythm play.  
Tech & dependencies
  - **Tech:** Finite-state machines for enemy types keyed to the beat clock (telegraphs/attacks on-beat), with spawns on fixed timers independent of the beat, tuned per type from data (custom C++).
  - **Prerequisites:** G1 (tick), G2 (enemy data), G6 (beat clock).
  - **Feeds into:** M2G10-class boss behaviour; the beat-flavoured combat.

*M2:*

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling to hold the frame + audio-latency budget under a full gesture queue and combat VFX, ASan on MSVC,

memory pools, and validation on-device (Pixel 9a, Native Android); the Firebase/Linux-server anti-cheat seam set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable and a tight audio-to-input latency budget.

- **G2 ★ Content Editor — Core (Music Importer)** — import a track, detect its BPM and auto-generate the beat map that aligns the player's attacks and the enemy AI to the song.

Tech & dependencies

- **Tech:** A tool that ingests a user track, runs the M1G6 BPM/onset analysis, previews and lets the user confirm/adjust the tempo grid, and serializes the beat map to JSON for the game to consume.
- **Libraries/Software:** Dear ImGui, the project's audio/BPM module (FMOD/OpenAL + onset detection), nlhmann/json.
- **Prerequisites:** M1G6 (BPM analysis), G2 (data model).
- **Feeds into:** Player-imported music replayability; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (gesture parsing, BPM detection, data loading) and packaging via GitHub Actions.  
**Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G9 Audio — Advanced** — the **dynamic music** (traditional Chinese + electronic fusion), adaptive layers and the on-beat finishing-move audio flair.



#### Tech & dependencies

- **Tech:** FMOD/OpenAL adaptive layers keyed to combat state, DSP effects and mixing buses, and beat-synced stingers for finishing moves and on-beat flair.
  - **Prerequisites:** M1G6 (audio system + beat clock).
  - **Feeds into:** The showcase audio identity; the on-beat reward feedback.
- **G8 Firebase — Accounts, Leaderboards & Analytics** — accounts, the **server-side anti-cheat** validation, and the currency/unlock/cosmetic store + leaderboards.

#### Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for currency/unlock/cosmetic data + leaderboards with server-side validation so clients cannot forge progression, optional Analytics, via the REST API from C++.
  - **Prerequisites:** G2 (progression/economy data model).
  - **Feeds into:** The anti-cheat guarantee; the shop and leaderboards.
- **G13 Gameplay Systems II** — the **ability-unlock / shop / currency** economy and the combo/Qi progression.

#### Tech & dependencies

- **Tech:** C++ systems over the entity/data model for move unlocks, the post-stage shop (moves, cosmetics, tracks bought with stage-earned currency) and the combo→Qi→finishing-move loop, all balancing-data-driven.
- **Prerequisites:** M1G2 (move/economy data), M2G2 (config authoring).
- **Feeds into:** The progression + replayability at showcase; T2 move expansion.

## Track Goals & Team Composition

**Recommended composition:** 6 CS + 1 Design + 2 Art (9) — the Chinese-mythology setting, per-element spell VFX, the enemy/boss roster and the cosmetics are a large, VFX-forward art surface, so two artists carry it; a single designer owns the gesture-combat moveset design (Five Elements), the optional-rhythm combo/Qi system, the stage/enemy design and the shop economy (rules: min 1 Design mandated, max 9 total,  $D+A \leq 3$ ; see [team-formation.pdf](#)). If the moveset/rhythm/economy design outgrows one designer, trade the 2nd artist for a 2nd designer. *Scope note (advisory):* gesture combat + BPM analysis + adaptive music + a music-import tool + Five-Elements movesets + 5 stages + bosses + a server anti-cheat store is a large surface — treat the gesture combat with ~2 elements, the beat clock + on-beat Qi reward, 2 stages and one boss as the M1/M2 floor, with all five elements, the music importer and the server store as the stretch. **BPM detection quality is the key audio-facing risk — prototype it early.**

**Track goals** — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the gesture-combat + optional-rhythm hook, the Five Elements, the combo/Qi loop and the mythology framing; primary benchmark e.g. Hi-Fi Rush, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving the **\*\*optional-but-rewarding beat hook\*\*** feels good — mashing works, on-beat feels better) · prototype in Unity
- M1D6 Systems & Economy Design (the Five-Elements moveset properties, the combo→Qi→finishing-move loop and the stage-currency shop — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (element/move balance, combo/Qi pacing and enemy tuning via the team's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):
  - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the Qi/combo/timer HUD, the on-beat cue and the element-select/shop screens.
- **UI/UX M2:**
  - M2U1 ★ Functional UI & Usability — combat HUD, element loadout, shop and music-import screens navigable unaided on the Native-Android target.
  - M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD + on-beat feedback tested with casual and rhythm-focused players on the Pixel 9a.
- **Art M1:** 2 artists justified (mythology setting + per-element VFX + enemy/boss roster)
  - M1A1 ★ Art Bible / Proof of Concept (the Chinese-mythology visual direction — palette, the Five-Elements visual language, readability in combat)
  - M1A2 Character Concepts & Model Sheets (the player, the enemy types and the bosses)
  - M1A3 Environment, Props & VFX Concepts (the stage settings + the per-element spell-effect language)
  - M1A4 UI Art Concepts & Typography (Qi/combo/timer HUD, element-select, shop)
  - M1A5 Animation Foundations (player casts per element, enemy/boss attacks, finishing-move flourishes)
  - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
  - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
  - M2A2 Character Animation — Extensive (elemental movesets, finishing moves, bosses)
  - M2A3 Environment Alive — Ambient Animation & Layering (the mythology stages)
  - M2A4 UI Art — States & Feedback (on-beat vs off-beat, Qi-full, finishing-move states)
  - M2A5 VFX — Critical Effects & Transitions (per-element spell VFX + the on-beat finishing-move flair — the core reward)
  - M2A6 Cutscenes & Storyboards (the mythology story beats)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — **load-bearing audio**: the traditional-Chinese + electronic dynamic tracks present & working in the prototype, with a tight, correctly-synced beat clock driving on-beat cues (a sync clause applies — the beat IS the mechanic).
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace/import files without recompiling) + reliable multi-channel playback and the BPM/beat-clock feed the gameplay consumes.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the full dynamic soundtrack (traditional Chinese + electronic) across the 5 stages, beat-synced to combat, with on-beat finishing-move flair ( $\geq 6$  cue types); a sync clause applies since audio is load-bearing.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised, adaptive layers, and the BPM-recognition support for imported tracks the build needs.
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the music importer), prototype demo, team post-mortem (wk14)

---

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-21.*